

Appello TAP del 02/07/2026

Avete a disposizione **2 ore (2 ore e 30 minuti per chi ha DSA/disabilità)**.
La prova è composta da:

- **Quiz** su AulaWeb (11 punti)
- Esercizio di **programmazione** (11 punti)
- Esercizi di **testing** ($6 + 5 = 11$ punti)

La somma totale è **33 punti**, ma è necessario arrivare almeno a **5 punti su ciascuna delle tre parti** per passare l'esame.

Suggerimento: leggete **tutto** il testo prima di buttarvi a implementare “cose”: la specifica dei test dell'esercizio 2 può aiutarvi a capire fin da subito cosa implementare (e cosa **NON** fare).

Per la consegna, caricate su AulaWeb la **solution “zippata”**, ovvero zippate la cartella che contiene il file `.sln` e i progetti, *dopo aver eseguito il Clean*. Codice che non compila o che non passa il `SanityCheck` (presente anche in versione `.txt` su AulaWeb) non verrà considerato. **Dopo aver caricato la solution, per sicurezza, scaricate il vostro zip, scompattatelo in un'altra cartella e verificate che compili tutto.**

Esercizio 1 (11 punti)

In un progetto *class library* implementare l'extension method

`TakeThenSkip<T>`

che, dati:

- una sequenza (*potenzialmente infinita*) di elementi di tipo `T`, `source`
- un intero strettamente positivo `take`
- un intero non negativo `skip`

restituisce una sequenza ottenuta ripetendo questo comportamento fino alla fine di `source`:

1. restituisce `take` elementi di `source`, se presenti (altrimenti, restituisce i rimanenti e poi termina)
2. scarta i successivi `skip` elementi di `source`, se presenti
3. ricomincia dal punto 1

Per esempio, se

```
source == [0, 1, 2, 3, 4, 5, 6, 7, 8, 9]
```

allora

```
source.TakeThenSkip(2, 3)
```

restituisce la sequenza

[0, 1, 5, 6]

perché vengono restituiti 0 e 1, scartati 2, 3 e 4, restituiti 5 e 6, scartati 7, 8 e 9.

Se `take <= 0`, il metodo dovrà sollevare `ArgumentOutOfRangeException` **immediatamente**, senza che sia necessario enumerare la sequenza restituita.

Se `skip < 0`, il metodo dovrà sollevare `ArgumentOutOfRangeException` **immediatamente**, senza che sia necessario enumerare la sequenza restituita.

Se `skip == 0`, il metodo non scarta nulla e restituisce tutti gli elementi di `source`.

Vincoli

- Il metodo non deve enumerare l'intera sequenza se non necessario.
- La sequenza `source` deve essere enumerata **una sola volta** per ogni enumerazione della sequenza restituita (quindi, **non** potete usare `Take` e `Skip` standard).
- Il metodo deve funzionare anche se `source` è infinita.

Sanity check

Lo scopo di questo test è solo verificare che la *signature* del vostro extension method sia abbastanza simile a quanto atteso da permettere di compilare ed eseguire i test.

```
[Fact]
public void SanityCheck() {
    IEnumerable<int> x = Array.Empty<int>().TakeThenSkip(2, 3);
    _ = new List<string> { "a", "b" }.TakeThenSkip(1, 0);
    _ = new double[] { 3.14, 2.71 }.TakeThenSkip(10, 100);
}
```

Ricordate che gli extension method devono essere `static` dentro una classe `static`.

Esercizio 2 (6 punti)

Implementare, usando xUnit, i seguenti test relativi all'esercizio 1:

1. Verificare che invocando il metodo con `take == 0` venga lanciata immediatamente l'eccezione `ArgumentOutOfRangeException`.
2. Verificare che invocando il metodo con `skip == -1` venga lanciata immediatamente l'eccezione `ArgumentOutOfRangeException`.
3. Dato `source` l'array vuoto di interi, `take == 2` e `skip == 3`, verificare che il risultato sia la sequenza vuota.
4. Dato `source` l'array [0, 1, 2, 3, 4, 5, 6, 7, 8, 9], `take == 2` e `skip == 3`, verificare che il risultato sia la sequenza [0, 1, 5, 6].

5. Dato `source` la lista `["qui", "quo", "qua"]`, `take == 1` e `skip == 0`, verificare che il risultato sia la sequenza `["qui", "quo", "qua"]`.
6. Dato `source` l'array `[10, 20, 30, 40, 50]`, `take == 10` e `skip == 100`, verificare che il risultato sia la sequenza `[10, 20, 30, 40, 50]`.

Per i test 1 e 2, fate attenzione: l'eccezione deve essere sollevata già dalla chiamata a `TakeThenSkip`, non solo quando si prova a enumerare la sequenza restituita.

Esercizio 3 (5 punti)

Implementare test aggiuntivi che verifichino il comportamento dell'esercizio 1 rispetto a esecuzione differita, singola enumerazione e sequenze infinite.

In particolare, scrivere test che verifichino che:

1. dopo

```
var result = source.TakeThenSkip(2, 3);
```

la sequenza `source` **non** sia ancora stata enumerata;
2. dopo aver enumerato una volta il risultato, per esempio con `result.ToList()`, la sequenza `source` sia stata enumerata **una sola volta**;
3. usando come `source` la sequenza infinita dei numeri interi non negativi `[0, 1, 2, 3, ...]`, con `take == 2` e `skip == 3`, i primi 8 elementi del risultato siano

```
[0, 1, 5, 6, 10, 11, 15, 16]
```

Suggerimento: per i primi due test potete creare una vostra sequenza che incrementa un contatore quando il metodo `GetEnumerator()` viene chiamato.